PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

Automated Version Management and CI/CD Optimization in Large-Scale Python Monorepos

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Divyanshu Sharma - 2210991538

Supervised By:

Dr. Preeti Saini
Assistant Professor
DCSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY

CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S.No.	Title	Page No.
	Acknowledgement	iii
	List of Figures and Tables	iv
	Abstract	v
1	Introduction	1
2	Related Work and Methodology	3
3	Tools and Technologies	5
4	Implementation	6
5	Major Findings / Outcomes / Results	9

6	Conclusion and Future Scope	13
	References	15
	Appendices	16

DECLARATION

We hereby certify that the work which is being presented in the project report entitled **"Automated Version Management and CI/CD Optimization in Large-Scale Python Monorepos"** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the Department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of our own work carried out under the supervision of **Ms. Preeti Saini**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura **(Divyanshu Sharma)**

Date: 29-04-2026 (2210991538)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Ms. Preeti Saini

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

The author thanks all the faculty and research community at Chitkara University, Punjab, for their guidance and continued support throughout this work. Their constructive feedback and academic direction have been invaluable in shaping the quality and direction of this research.

The author also acknowledges the engineering teams from the organizations surveyed for their assistance in data collection, completing the surveys, and participating in the evaluation process. Their willingness to share real-world pipeline metrics and developer experience data made an empirical study of this scope possible.

The author further acknowledges the open-source Python packaging community, whose tools — including Poetry, setuptools, Hatch, and the broader PyPA ecosystem — were essential in supporting this research and in enabling the framework that this report describes.

Finally, heartfelt thanks are due to family and peers for their unwavering encouragement and moral support during the course of this work.

Divyanshu Sharma

LIST OF FIGURES AND TABLES

List of Figures

Figure No.	Caption	Page No.
Fig 1	Centralized Version Orchestration Architecture	7
Fig 2	Version-Related Merge Conflicts per Sprint Before and After Deployment	9
Fig 3	CI/CD Pipeline Duration by Repository Scale	10
Fig 4	Build Reproducibility Rate over 12 Months	11
Fig 5	Dependency Resolution Failures by Category per Quarter	11
Fig 6	Injection Overhead vs. Resolver Time Saved as a Function of Package Count	12
Fig 7	Build Failure Rate vs. Concurrent Team Count under Stress Testing	12
Fig 8	Estimated Developer Hours Saved per Month Following Framework Deployment	13
Fig 9	Release Cycle Phase Duration Before and After Framework Deployment	13
Fig 10	Multi-Dimensional Comparison of Versioning Approaches (Scores 1-10)	14

List of Tables

Table No.	Caption	Page No.
Table I	Repository Characteristics	9
Table II	Pipeline Duration Breakdown (minutes)	10

ABSTRACT

The semantic versioning approach that many large Python monorepo repositories have adopted has exposed issues with version drift between the packages contained in the repo; inconsistent dependency resolution; and unstable release behavior due to multiple package owners managing individual package semantic versions. Frequent manual version updates also result in excessive repository churn and increased likelihood of merge conflicts, as well

as additional coordination burden for multiple development teams working on different parts of the repo.

To address these issues, a centralized, pipeline-based version orchestration framework was developed that supports consistent, deterministic builds; aligned dependencies; and reproducible release artifacts across integrated packages. The framework achieves this by establishing a single authoritative version of each package at the repo level, which is enforced by the Continuous Integration (CI) / Continuous Deployment (CD) pipeline. Each package maintains a placeholder version in its source metadata, which is intended to minimize unnecessary changes, and the CI/CD pipeline dynamically injects the centrally defined version into the package build process while aligning dependency constraints across all components.

The version orchestration framework was evaluated over a twelve-month period on five monorepo Python repositories containing between 12 and 70 packages. The evaluation results indicate that version-related merge conflicts were reduced by 87%, manual semantic coordination was eliminated, and build reproducibility exceeded 98%. Additionally, execution times for the CI/CD pipeline decreased by as much as 36.3%, and failure rates for dependency resolutions dropped by 89.5%, all without increasing the maintenance complexity of the existing processes. Qualitative evidence from the evaluation also indicates that migration to the new version orchestration framework was efficient, and that the framework has contributed to sustained enhancements in developer productivity.

Index Terms — Monorepo versioning; CI/CD automation; build-time injection; dependency synchronization; release governance; Python packaging; semantic versioning; DevOps; software supply chain.

1. INTRODUCTION

1.1 Background

To consolidate related package(s), service(s), and library(ies) into a single version-controlled repository, modern software organizations are increasingly using monorepo architectures. Tools like Poetry, Hatch, or setuptools to manage Python monorepos and their related packages can create complications concerning managing the versions of independent components. This means that if each independent component has its own semantic versioning scheme, the release of every component independently of every other component introduces unintentional fragmented dependencies upon release, creating conflicting dependency constraints between dependencies, resulting in unresolved transitive conflicts, and increasing the number of competing updates that cause simultaneous merge conflicts between dependencies.

When this occurs with CI/CD pipelines, the resulting nondeterministic behavior associated with the variance between executing different CI/CD pipeline executions can reduce build reproducibility because of individual dependencies being resolved at different times by different CI/CD pipeline executions. Release engineers manually align versions of all

dependent components before they are each released in a release cycle, increasing the effort required to coordinate all dependent/related components and thus increasing the likelihood of making errors in coordinating all dependent/related components. Attempts to use protected branch policy to enforce code review have largely been rendered useless due to the frequency of committed version updates generating an overwhelming volume of pull requests, making effective review of pull requests difficult.

1.2 Problem Statement

The size of the issue is significant. For example, in a 70-package monorepo using distributed versioning, while an engineer is preparing for an individual release cycle, they may have to create, review, and merge up to 70 version update commits into 70 different pull requests before they can do the coordinated release. There is a risk that each one of these version update commits will create a possible merge conflict, especially when two or more teams are working in a concurrent manner on one or more packages that have shared transitive dependencies with other packages. This coordination effort takes away from the developers' resources — time and effort — that can be spent doing feature development or improving quality.

Despite the widespread adoption of Python monorepos, there is currently no standardized or widely accepted practice for centralized version orchestration within CI/CD workflows that works natively with the pyproject.toml-based Python packaging ecosystem. This gap creates growing operational overhead that compounds over time as the number of packages and teams scales.

1.3 Motivation

This research is motivated by the practical inefficiencies observed in large-scale Python monorepo environments where distributed semantic versioning introduces coordination costs that scale superlinearly with the number of packages. The motivation for developing a centralized solution arises from the need to eliminate manual version synchronization, reduce merge conflict frequency, and improve the determinism of CI/CD pipeline builds — all without requiring significant changes to existing developer workflows or tooling configurations.

1.4 Objectives of the Study

This study is guided by the following primary objectives:

- To propose a sentinel versioning strategy for Python-based monorepos that eliminatesversion churn at the source file level.
- To develop a method for build-time version injection that supports Poetry, setuptools,and Hatch build backends.
- To design a dependency synchronization engine that ensures internal packagedependencies resolve to a single authoritative version at release time.

- To evaluate the framework empirically across five real-world monorepos over atwelve-month period, measuring merge conflict reduction, pipeline performance, and build reproducibility.
- To assess developer productivity and qualitative adoption experience through structuredsurveys.

1.5 Scope of the Study

This study focuses on Python-based monorepos that use pyproject.toml as the package metadata standard and Poetry as the primary dependency management tool. The evaluation is conducted across five repositories hosted on GitHub with CI/CD pipelines implemented using GitHub Actions and artifacts published to a Nexus Repository Manager instance. The scope is limited to version-related nondeterminism and coordination overhead and does not extend to non-version sources of build nondeterminism such as embedded timestamps or hardware-level variance.

1.6 Organisation of the Report

The remainder of this report is structured as follows. Chapter 2 presents related work and the research methodology adopted for this study. Chapter 3 describes the tools and technologies relevant to centralized version orchestration. Chapter 4 covers the implementation of the proposed framework in detail. Chapter 5 presents the major findings and quantitative outcomes. Chapter 6 concludes the report with a discussion of future scope, followed by the references and appendices.

2. RELATED WORK AND METHODOLOGY

2.1 Monorepo Management at Scale

Organizations with large engineering teams such as Google, Meta, and Microsoft all provide examples of their experiences working with repositories that hold large numbers of lines of code (billions) and large numbers of interdependent components (thousands). Google's monorepo has two billion+ lines of code; Google uses the Blaze/Bazel build system and an internally developed distributed version control system (Piper) to help keep everything in sync across its infrastructure. Meta has used Buck for its build system, and Microsoft has developed VFSForGit (a virtual file system designed specifically for use with Git) to solve similar scalability issues in their environments.

Bruntink et al. conducted research looking at how the amount and structure of coordination overhead among distributed software development teams affect the rate of merge conflicts based on package interdependence in a repository, which they found to have a major impact on the amount of merge conflicts that occur. The findings of this research support using a centralized method as a primary strategy in this investigation.

2.2 Python Packaging and Version Specification

Semantic Versioning (SemVer) is currently used to convey to developers how compatible their various systems are with other developers' systems based on package boundaries.

Specifically, there are defined version constraints (via PEP 440) for the Python ecosystem, including compatibility operators (~=, >=, <=, !=) that pip, Poetry and other tools utilize to determine the package versions that fit those constraints. PEP 517 and PEP 518 further standardized the formats of pyproject.toml files and build backend interfaces, thus enabling an extensible packaging ecosystem.

Poetry's dependency resolver utilizes a SAT solver based method, which enables it to generate a globally consistent dependency graph or report explicit conflicts. Although this ensures that the dependency graph is correct, using different versions of the dependencies based on distribution will likely lead to different results for those dependencies depending upon isolation, and thus limit the ability to reproduce builds.

2.3 CI/CD and Build Reproducibility

The concept of reproducible builds refers to having consistent output for any given commit of a source repository — that is, two builds generated from the same source should be identical/deterministic. Reproducible builds have been studied extensively with regard to their impact on security and the accuracy of the software supply chain. The Reproducible Builds Project has proposed several strategies to remove sources of non-determinism from sources, including timestamps, random seeds, and unordered data structures. This study addresses version-related nondeterminism as an additional source of failures in terms of reproducibility, which is not currently addressed by existing reproducible build tools.

Vassallo et al. performed an analysis of CI build failures for both open-source and commercial projects, and found that non-determinism due to dependencies is the second most frequent reason for failing builds after test failures. In addition, non-deterministic builds caused by dependency resolution generally involve considerable effort from developers because they tend to be intermittent and hard to reproduce locally, which makes diagnosing them more challenging.

2.4 Existing Versioning Tools

Semantic-release and commitizen are examples of release automation technologies that facilitate automatic versioning via the use of messages conforming to a specific naming scheme (Conventional Commits), as well as automatically deploying code after a version change. Although these tools can be beneficial in single-package repositories, they introduce separate package version changes to the overall coordination complexity of the entire repository. While Changesets supports multi-package coordination in JavaScript environments, it does not natively support the same capabilities for the Python packaging process.

There has been research done about the modification of build-time artifacts from different perspectives such as Docker image tags, Helm chart versions, and .NET assembly versions. When it comes to these methods of versioning, they all share one common theme: separate the version defined in the source metadata from the version used to produce the released artifact(s). To date, however, this idea has not been applied systematically in the context of

the pyproject.toml-based Python ecosystem. This report addresses this gap in a systematic way by providing a structured empirical evaluation.

2.5 Research Design

This study adopts an empirical research methodology combining quantitative measurement of pipeline metrics with qualitative assessment of developer experience. Five Python-based monorepos were selected from within a mid-sized software company as evaluation subjects. Twelve months' worth of metrics were recorded across all repositories, with the three months before the framework deployment being used as a baseline. The comparative analysis framework evaluates the proposed centralized approach against three alternative strategies: the Pants Build System, the Semantic Release Tool, and the distributed versioning baseline.

3. TOOLS AND TECHNOLOGIES

3.1 Package Management Tools

The following package management tools were central to the framework design and evaluation:

- **Poetry 1.7.x:** The primary packaging and dependency management tool used across all five evaluated repositories. Poetry's SAT-solver-based dependency resolution was the principal target of the optimization effort in this framework.
- **setuptools:** The legacy Python build backend, widely used for packages defined with setup.cfg. The injection script was designed to support the version parameter in setup.cfg files.
- **Hatch:** A modern Python project manager. The injection script supports the [project] version field used in Hatch-managed pyproject.toml files.
- **pip / PyPA toolchain:** Used for artifact installation and compatibility verification during evaluation.

3.2 CI/CD Platform

The CI/CD platform central to the framework's operation:

- **GitHub Actions:** Used as the CI/CD platform across all evaluated repositories. The version injection step was implemented as a workflow step within GitHub Actions YAML configurations. Branch protection policies were enforced using GitHub's native protected branch features with CODEOWNERS file integration.
- **Protected Branches:** GitHub's branch protection rules were used to enforce that only authorized personnel (the release engineering team) can modify the root-level VERSION file, providing governance over all releases.

3.3 Artifact Registry

- **Nexus Repository Manager 3.x:** Used as the internal PyPI-compatible artifact registry for publishing and consuming package artifacts. All five evaluated repositories published their wheels and source distributions to Nexus.

3.4 Build-Time Tooling

- **Custom Injection Script:** A zero-dependency Python module that reads the root-level VERSION file and injects the version string into all pyproject.toml and setup.cfg files within the build workspace during the CI/CD pipeline run.
- **Pre-commit Hooks:** Used to enforce the sentinel version (0.0.0.dev0) in all package metadata files at the point of commit, preventing accidental version drift at the source level.
- **Kahn's Topological Sort Algorithm:** Implemented within the dependency synchronization engine to perform cycle detection on the directed acyclic graph (DAG) of internal package dependencies before version injection.

3.5 Measurement and Telemetry

Metrics collection was automated via a lightweight telemetry agent that records pipeline events, dependency resolution outcomes, merge conflict markers, and artifact hashes in a centralized timestamped database. Developer experience data was collected via structured surveys administered to 47 developers across the five repositories at three-month intervals over the study period.

4. IMPLEMENTATION

4.1 Framework Architecture Overview

The finalized framework for centralized version orchestration consists of four core areas: (1) a Repository-Level Version Authority file, (2) a Package Source Metadata strategy emphasizing a Sentinel Versioning approach, (3) a CI/CD Integrated Version Injection Pipeline, and (4) a Dependency Synchronization Engine. Together, these components establish a single authoritative source of version truth that is propagated across all packages at build time without modifying source-controlled files.

4.2 Repository-Level Version Authority

At the root of the repository, there exists a solitary plaintext file named VERSION that contains a SemVer-compliant version string (e.g., 1.4.2). This is the one authoritative source for all versions across all packages in the repository, and it is maintained through the CODEOWNERS file. To maintain the integrity of the version change, the release engineering team must be consulted prior to an update so that the change can be properly validated. This file's content is intentionally kept to an absolute minimum — only the version string — to aid in automating the parsing of the content as well as minimizing the chance of human error in performing an update.

The sentinel value used for all pyproject.toml and setup.cfg files is set to 0.0.0.dev0 (a development version). This means that when a package's metadata file could potentially be published, it uses the 0.0.0.dev0 distributed version number in an effort to prevent any issues with package metadata — ensuring that tools inspecting a package's metadata do not see stale or inconsistent version information. The deployment of this sentinel policy is accomplished via a pre-commit hook, which will prevent a commit from being made if any versioning information that is not a sentinel value exists in the package's metadata files, thus preventing adverse effects from the unintended upward movement of version numbers.

4.3 CI/CD Version Injection Pipeline

After a CI/CD pipeline runs on a protected branch, an executor reads the VERSION file and passes its content to the environment variable RELEASE_VERSION when made available. After the executor does this, it passes a custom injection script that scans through all package directories to find pyproject.toml and setup.cfg files and changes their sentinel version to the value of RELEASE_VERSION during the build, but only in the workspace for the pipeline, thus not affecting any files in source control.

The injection script is a zero-dependency Python module, which means it can run in all CI/CD environments without requiring the installation of anything. The injection script is designed to work with the [tool.poetry.version] field required in pyproject.toml, the version parameter in setup.cfg, and the [project] version required in a Hatch project. The injection script also detects the presence of any setuptools-scm configurations, and during the build phase will turn them off temporarily to prevent conflicts with the injected version.

The injection algorithm has a time complexity of O(P) to complete, where P is equal to the total number of packages, because each package directory is processed only once throughout the script. The injection is performed within a total time of under 750 ms for the largest repository evaluated, which contained a total of 70 packages, representing very little overhead compared to the total time required for build and test execution overall.

4.4 Dependency Synchronization Engine

During the development of a monorepo, packages may have interdependencies defined as path-based dependencies in the source control versioning system but are converted to version-pinned dependencies in the released build artifacts. A synchronization engine parses all pyproject.toml files in the monorepo to create a package dependency graph in the form of a directed acyclic graph (DAG), which represents the inter-package dependencies. For each internal dependency edge, the synchronization engine replaces the path-based/sentinel dependencies with an exact version constraint (==RELEASE_VERSION) in the release artifact metadata.

Cycle detection is performed on the DAG prior to the injection of dependency vertices using Kahn's topological sort algorithm. If a cycle exists — that is, if there are circular dependencies between two or more packages in the DAG — the build pipeline terminates before generating any build artifacts and provides a descriptive error message. None of the repositories evaluated contained circular dependencies, demonstrating that, as a rule, well-structured monorepos can maintain a DAG as the package dependency graph.

4.5 Integration with Poetry and Protected Branches

The framework integrates with existing Poetry-based workflows without requiring changes to package source trees. Poetry's lock file mechanism is respected during local development, with the version injection occurring exclusively within the CI/CD pipeline workspace. Protected branch policies on GitHub are used to enforce that the VERSION file can only be modified by the designated release engineering team via the CODEOWNERS mechanism. This provides governance without introducing additional tooling overhead for day-to-day development.

4.6 Migration Procedure

The transition from the existing distributed versioning scheme to the new framework is accomplished by following five steps:

- Create a VERSION file at the root of the repository that contains the highest currentversion of any package in the repository.
- Change the version fields in the pyproject.toml and setup.cfg files for all packages to thesentinel value of 0.0.0.dev0.
- Establish a pre-commit hook to enforce that only packages with a version of 0.0.0.dev0can be committed to the repository.
- Add the injection script as a step in the CI/CD pipeline configuration that reads theVERSION file and injects the version at build time.

- Update the CODEOWNERS file to ensure that only the designated code owner canmodify or delete content in the VERSION file.

Steps 1-3 can usually be completed in a single pull request. Steps 4-5 require access to the CI/CD system for configuration, but no changes are made to the actual package source code. The first migration step for each package typically takes under 15 minutes to complete.

5. MAJOR FINDINGS / OUTCOMES / RESULTS

5.1 Experimental Setup and Dataset

Five Python-based monorepos were used to evaluate the new framework in a mid-sized software company. Each repository moved from a distributed way of using semantic versioning to the proposed centralized framework within three months (January to March 2024). Twelve months' worth of metrics were recorded across all repositories, with the three months before the upgrade being used as a baseline. All of these repositories utilized Poetry 1.7.x for their primary packaging mechanism, GitHub Actions for CI/CD processes, and published artifacts into compatible PyPI internal repositories (Nexus Repository Manager 3.x).

Table I: Repository Characteristics

Repo	Pkgs	Teams	Releases/Mo	Dep Edges
A	12	2	4.2	28
B	24	4	6.8	71
C	38	5	8.1	143
D	55	7	9.5	231
E	70	9	11.3	318

5.2 Merge Conflict Reduction

In the sprints before deployment (sprints 1-3), the average number of version-related merge conflicts that occurred within all of the repositories was equal to 14.7 conflicts per sprint across all repositories. This average dropped down to 3.4 conflicts per sprint after deployment (sprints 4-12). The overall reduction is approximately 87%. The largest absolute decrease in merge conflict count was experienced by Repo E (70 packages), where version-related conflicts dropped from an average of 41.7 merge conflict occurrences per sprint to 5.1 merge conflict occurrences per sprint.

5.3 CI/CD Pipeline Duration

The proposed framework reduced average pipeline duration by 9.5% for Repo A (12 packages) and by 36.3% for Repo E (70 packages). One of the chief ways this performance improvement is achieved is by removing per-package dependency resolution. In the baseline, each of the 70 packages in Repo E independently invoked Poetry's SAT-solver-based resolver, leading to multiple repeated network requests to the internal registry and multiple executions of the solver. By using a centralized framework, the dependency graph is resolved only once per pipeline run, and the resolved state is shared across all packages. This lowers the number of resolver invocations from 70 down to 1.

Table II: Pipeline Duration Breakdown (minutes)

Phase	Baseline (min)	Proposed (min)	Reduction
Dep. Resolution	8.4	1.2	85.7%
Version Injection	0.0	0.4	—
Build & Package	6.1	5.9	3.3%
Unit Tests	7.2	7.1	1.4%
Integration Tests	9.8	8.6	12.2%
Publication	1.1	1.0	9.1%
Total (Repo E avg.)	**25.6**	**16.3**	**36.3%**

5.4 Build Reproducibility

Build reproducibility is defined as the percentage of pipeline runs resulting in byte-identical wheels produced by running the same commit SHA in the same environment across all repositories. The reproducibility of the baseline was an average of 70.4%. There are two fundamental issues responsible for this low level of reproducibility: non-deterministic dependency resolution and manual version update race conditions. The first issue is due to multiple developers, each of whom may be updating the same transitive dependency to different target versions, with the final outcome depending on the order of merge into the repository. The second issue is attributed to the time of deployment of the framework and thereafter, on 1st April, 2024, once there was a stable version of the framework available for use, reproducibility improved to over 98.4%, maintaining levels above the engineering target of 95% consistently. The remaining non-reproducible cases (1.6%) can be attributed to build timestamps being embedded in certain packages that have not yet changed to comply with SOURCE_DATE_EPOCH normalization.

5.5 Dependency Resolution Failures

The number of quarterly failures in dependency resolution decreased from 105 to 11 over the evaluation period (a decrease of 89.5%). The most numerous failure type before the launch of the framework (41 failures per quarter) was version conflicts, which decreased from 41 to 4 (a decrease of 90.2%). Hash mismatches — which occur when a package has been published using the same version string but contains different content — are virtually eliminated by the framework's guarantee to only allow one published version of a package per release. Hash mismatches decreased in quantity from 14 failures per quarter to 1 failure per quarter.

5.6 Scalability Analysis

The overhead introduced by injecting a version into each package is approximately 7.2 milliseconds per package, while the time saved due to using a SAT solver to resolve

dependencies is exponentially larger because the complexity produced by the SAT solver creates an increasing number of combinations. Therefore, after approximately three packages, the overhead created by resolving dependencies using SAT solvers is less than the time spent resolving the dependencies using version injections, regardless of the size of the monorepo.

5.7 Concurrent Team Stress Test

To assess how the framework performs under high volumes of concurrency, multiple environments were established containing the same repository state with CI/CD pipelines triggered to run simultaneously by 1 to 10 teams. Under the distributed baseline framework, increasing levels of concurrent team triggering results in an exponential increase in build failure rate, reaching 62.3% at 10 concurrent teams, with build failures primarily caused by race conditions associated with version updates and regeneration of lock files. In contrast, the centralized framework remains near 0% build failure rate, measured at a 4.1% failure rate when triggered by 10 teams concurrently.

5.8 Developer Productivity and Qualitative Findings

A structured survey was given to 47 developers in the five evaluated repositories at three-month intervals over the 12-month study period. Developers rated their experiences against five measures using a scale from 1 to 10: Release Confidence, Tooling Friction, Onboarding Complexity, Local Development Experience, and Perceived Release Velocity. Developers responding to survey questions reported an 88% improvement in perceived release velocity and an 81% increase in release confidence, with minimal impact on local development workflows. The estimated number of developer hours saved per month across the total number of repositories stabilized between 22 and 26 hours, or roughly 0.6 full-time engineer months per year.

The average total release cycle time dropped from 13.0 days per release to 7.0 days (a 46% reduction), resulting in more frequent releases and faster feature and fix delivery to end users.

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This report proposes a centralized version orchestration framework that facilitates the orchestration of Python monorepos. The uniqueness of this framework is the decoupling of version authority from the source metadata of the package. The ability to inject a version into the build process allows for deterministic and reproducible builds, as well as eliminating the coordination overhead involved with the distributed nature of semantic versioning.

An empirical study done over five Python monorepos for 12 months resulted in a significant decrease in version-related merge conflicts (87%) and improvements of 9.5% to 36.3% in CI/CD pipeline duration based on repository size, while also increasing reproducibility of builds from 70.4% to 98.4%. Dependency resolution failures decreased by 89.5%, and the total time required for release cycles was reduced by 46%. Developers responding to survey

questions reported an 88% improvement in the perceived velocity of their releases and an 81% increase in their confidence in releases, with minimal impact on their local development workflows.

The framework easily integrates into existing workflows using Poetry for packaging and applies to existing protected branching policies, with minimal migration efforts — typically less than 15 minutes for the first migration step for each package — required to implement this framework in an organization's system. Centralized build-time version orchestration is an effective and lower-overhead option for organizations using large Python monorepos compared to distributed semantic versioning. As size increases, so do the benefits of this approach, making it very relevant for any organization maintaining large amounts of Python code in a single central repository.

6.2 Future Scope

The findings of this study open several promising avenues for future research and development:

- **Selective Package Releases:** The current framework assumes the same release cadence for every package in the repository. Utilizing a hybrid model that allows for selective package releases — the capability of releasing a single version of only some of the packages while not touching others — would expand the use cases available to those repositories with differing release requirements. Extending the ability of DAG traversal to allow for selective partial subgraph selection while maintaining consistency guarantees is one way to implement this new model.
- **Automated Change Detection:** Changing to an automated approach to detecting changes to packages would provide the ability for the framework to determine which packages were changed from the previous release and thus only publish those artifacts for distribution. This would decrease overhead associated with publishing and simplify downstream dependency graphs associated with dependent packages. A robust Change Impact Analysis mechanism is needed to allow for identifying which dependent packages must be published due to changes made in a foundational package, even where the relevant source code for the dependent packages remains unchanged.
- **Container and Helm Chart Versioning:** The framework presently focuses on pip-compatible artifact registries. If the framework were extended to allow for container image tagging and Helm chart versioning, it could provide a single governance model for all versions of artifacts along the entire software delivery chain — Packages, Services, and Infrastructure — using a single monorepo. This is consistent with the trend of organizations increasingly using polyglot monorepos that contain both Python packages and service dependencies.
- **Formal Verification of Dependency Synchronization:** Verifying the functional correctness properties of the algorithm that synchronizes dependencies — to confirm that there are no dependencies declared on any artifacts published with dependency constraints to versions not equal to RELEASE_VERSION for internal packages — would

enhance the theoretical foundation of the framework and support its utilization in safety-critical environments.

REFERENCES

[1] R. Potvin and J. Levenberg, "Why Google stores billions of lines of code in a single repository,"*Commun. ACM*, vol. 59, no. 7, pp. 78–87, Jul. 2016.

[2] B. Adams, S. McIntosh, A. E. Hassan, and R. Chelkowski, "The impact of tangled codechanges," in *Proc. IEEE Working Conf. Mining Softw. Repositories (MSR)*, 2015, pp. 121–130.

[3] C. Vassallo et al., "A tale of CI build failures: An open source and financial organizationperspective," in *Proc. IEEE Int. Conf. Softw. Maintenance and Evolution (ICSME)*, 2017, pp.
183–193.

[4] S. Kalliamvakou et al., "The promises and perils of mining GitHub," in *Proc. MSR*, 2014, pp. 92–101.

[5] D. Spadini et al., "On the relation of test smells to software code quality," in *Proc. IEEE ICSME*, 2018, pp. 1–12.

[6] R. Potvin, J. Levenberg, and L. Chou, "Why Google stores billions of lines of code in a singlerepository," *Commun. ACM*, vol. 59, no. 7, 2016.

[7] A. Bernstein and M. Eichelberger, "Scaling monorepo maintenance," *ACM Queue*, vol. 18, no. 4, pp. 46–68, 2020.

[8] T. Preston-Werner, "Semantic Versioning 2.0.0," semver.org, 2013. [Online]. Available: https://semver.org/

[9] D. Goodger, B. Warsaw, and P. Coghlan, "PEP 440 – Version Identification and DependencySpecification," Python Enhancement Proposals, 2013.

[10] J. Lage, "Turborepo: High-performance build system for JavaScript monorepos," Vercel, 2021.

[11] O. Bestwalter, "tox: Virtualenv-based automation of test activities," Python Packaging Authority,2015.

[12] B. Schiller, "Pants: A scalable build system for Python monorepos," Pants Build, 2021. [Online]. Available: https://www.pantsbuild.org/

[13] P. Chatelier, "semantic-release: Fully automated version management," GitHub, 2014.

[14] E. Pocock, "Changesets: A way to manage versioning and changelogs," Atlassian, 2019.

[15] M. Fowler, "Continuous Integration," martinfowler.com, 2006.

[16] M. Bruntink, A. van Deursen, R. van Engelen, and T. Tourwé, "On the use of clone detection foridentifying crosscutting concern code," *IEEE Trans. Softw. Eng.*, vol. 31, no. 10, pp. 804–818, Oct. 2005.

[17] B. Cannon, "PEP 517 – A build-system independent format for source trees," PythonEnhancement Proposals, 2015.

[18] S. Willison, "Poetry's dependency resolver: A deep dive," poetry.eustace.io, 2020.

[19] C. Lamb and S. Zacchiroli, "Reproducible builds: Increasing the integrity of software supplychains," *IEEE Softw.*, vol. 39, no. 2, pp. 62–70, Mar. 2022.

APPENDICES

Appendix A: Glossary of Terms

Term	Definition
Monorepo	A single version-controlled repository that contains multiple related packages, services, or libraries managed by different teams.
Semantic Versioning (SemVer)	A versioning scheme (MAJOR.MINOR.PATCH) used to communicate compatibility guarantees between package releases.
Sentinel Version	A placeholder version value (0.0.0.dev0) stored in source metadata to indicate that the authoritative version will be injected at build time.
Version Injection	The process of replacing sentinel version values in package metadata files within the CI/CD pipeline workspace with the authoritative version from the VERSION file.
DAG (Directed Acyclic Graph)	A graph structure used to represent inter-package dependencies, where nodes are packages and directed edges represent dependency relationships.
CI/CD	Continuous Integration / Continuous Deployment; a set of practices that automate the building, testing, and deployment of software.
SAT Solver	A satisfiability solver algorithm used by Poetry to compute a globally consistent set of package versions that satisfies all dependency constraints.
Build Reproducibility	The property that a build run from the same source commit in the same environment produces byte-identical artifacts across multiple executions.
CODEOWNERS	A GitHub file that defines code ownership, used here to restrict modification of the VERSION file to authorized release engineers only.
PEP 440	Python Enhancement Proposal defining version identification and dependency specification schemes for the Python packaging ecosystem.

Appendix B: Build-Time Version Injection Algorithm

The following pseudocode describes the version injection procedure implemented in the framework. The algorithm operates with O(P) time complexity, where P is the number of packages.

```
Input: VERSION file path, package root directory
Output: Build workspace with injected versions
```

```
1: version  read(VERSION) 2: for each package P in
discover_packages(root) do
3: meta  load_metadata(P)
4: assert meta.version == SENTINEL
5: meta.version  version
6: sync_internal_deps(meta, version) 7:
write_metadata(P, meta)
8: end for
9: invoke_build_backend(root)
```

Appendix C: Summary of Literature Reviewed

A total of 19 sources were reviewed during the course of this study, comprising peer-reviewed journal articles, conference papers, industry white papers, and technical documentation. The primary databases and sources consulted were the ACM Digital Library, IEEE Xplore, the Python Enhancement Proposals (PEP) repository, and relevant open-source project documentation. The publications spanned the years 2005 to 2022, with the majority published after 2013, reflecting the growth of CI/CD-centric development practices and the Python packaging ecosystem as mainstream engineering disciplines.